Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

Звіт

про виконання лабораторної роботи №7

«ФУНКЦІЇ»

Виконав:
студент 1 курсу
групи ФеП-11
Савлов Данило

Викладач:
асист. Васюта В. М.

Львів 2026

Мета: вивчити поняття та застосування функцій у мові програмування Сі.

Хід роботи

1. Створити функцію, яка перевіряє, чи введене з консолі число є простим.

```c
#include <stdio.h>
#include <stdbool.h>

/* Функція повертає true, якщо n є простим числом. */
bool is_prime(int n) {
    if (n < 2) {
        return false;
    }
    if (n == 2) {
        return true;
    }
    if (n % 2 == 0) {
        return false;
    }

    /* Перебираємо лише непарні дільники до кореня з n */
    for (int d = 3; d * d <= n; d += 2) {
        if (n % d == 0) {
            return false;
        }
    }
    return true;
}

int main(void) {
    int number;

    printf("Введіть ціле число: ");
    scanf("%d", &number);

    if (is_prime(number)) {
        printf("Число %d є простим\n", number);
    } else {
        printf("Число %d не є простим\n", number);
    }

    return 0;
}
```

Результат виконання програми для простого та складеного числа:

```
Введіть ціле число: 2
Число 2 є простим
```

```
Введіть ціле число: 55
Число 55 не є простим
```

2. Написати програму, котра рекурсивно вираховує суму чисел від Вашого порядкового номера в журналі до 100.

```c
#include <stdio.h>

#define UPPER_LIMIT 100

/* Рекурсивно рахує суму всіх цілих чисел від from до to включно.
   Рекурсія іде від верхньої межі вниз. */
long sum_range(int from, int to) {
    if (to < from) {
        return 0;
    }
    return to + sum_range(from, to - 1);
}

int main(void) {
    int journal_number;

    printf("Введіть свій номер у журналі: ");
    scanf("%d", &journal_number);

    long total = sum_range(journal_number, UPPER_LIMIT);

    printf("Сума чисел від %d до %d = %ld\n", journal_number, UPPER_LIMIT, total);

    return 0;
}
```

```
Введіть свій номер у журналі: 17
Сума чисел від 17 до 100 = 4914
```

3. Написати програму, котра знаходить розв’язок рівняння N*x + A = 0 методом половинного ділення. N – Ваш порядковий номер в журналі, A – Ваш вік (повних років). Розв’язок знаходиться в інтервалі [–100..100].

```c
#include <stdio.h>
#include <math.h>

/* Ліва частина рівняння: N*x + A */
double equation(double x, double n, double a) {
    return n * x + a;
}

/* Метод половинного ділення на відрізку [left, right].
   Повертає наближене значення кореня. */
double bisection(double left, double right, double eps, double n, double a) {
    double middle = left;

    while (right - left > eps) {
        middle = (left + right) / 2.0;
        double f_middle = equation(middle, n, a);

        if (f_middle == 0.0) {
            return middle;
        }

        /* Корінь лежить там, де функція змінює знак */
        if (equation(left, n, a) * f_middle < 0) {
            right = middle;
        } else {
            left = middle;
        }
    }
    return (left + right) / 2.0;
}

int main(void) {
    double n, a;
    const double left = -100.0, right = 100.0;
    const double eps = 0.001;

    printf("Введіть N (номер у журналі): ");
    scanf("%lf", &n);
    printf("Введіть A (вік у повних роках): ");
    scanf("%lf", &a);

    /* Перевірка: на кінцях відрізка знаки мають бути різні */
    if (equation(left, n, a) * equation(right, n, a) > 0) {
        printf("На відрізку [%g; %g] розв'язку немає\n", left, right);
        return 1;
    }

    double root = bisection(left, right, eps, n, a);
    printf("Розв'язок рівняння %g*x + %g = 0: x = %.3f\n", n, a, root);

    return 0;
}
```

```
Введіть N (номер у журналі): 17
Введіть A (вік у повних роках): 17
Розв'язок рівняння 17*x + 17 = 0: x = -1.000
```

Висновок: у ході лабораторної роботи я ознайомився з функціями в мові Сі та навчився виносити окрему логіку в самостійні блоки коду. Функцію is_prime використав для перевірки числа на простоту, рекурсивну функцію sum_range – для обчислення суми чисел до 100, а для методу половинного ділення розділив програму на дві функції: equation описує саме рівняння, а bisection виконує пошук кореня. Завдяки цьому формулу можна змінити, не чіпаючи алгоритм. Також зрозумів, що рекурсія зручна для задач, які зводяться до таких самих задач меншого розміру.